# Large Language Model-Brained GUI Agents: A Survey

Chaoyun Zhang[1]*, Shilin He[1]*, Jiaxu Qian[1], Bowen Li[2], Liqun Li[1], Si Qin[1], Yu Kang[1], Minghua Ma[1], Guyue Liu[4], Qingwei Lin[1], Saravan Rajmohan[1], Dongmei Zhang[1], Qi Zhang[1]
[1]*Microsoft* [2]*Shanghai Artificial Intelligence Laboratory* [3]*Peking University*

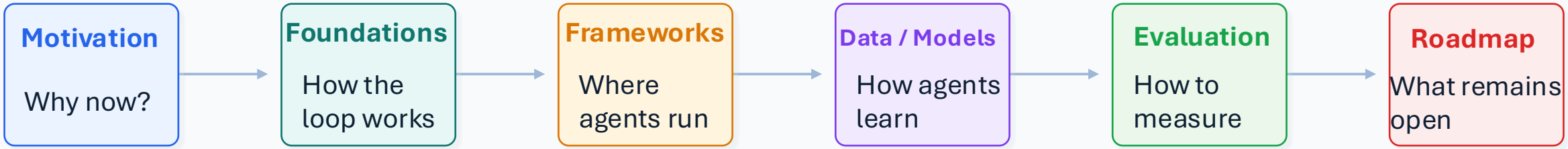
Cookbook | Reference | Roadmap

Transactions on Machine Learning Research 2025

Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

# What is the paper trying to map?

## Large Language Model-Brained GUI Agents: A Survey

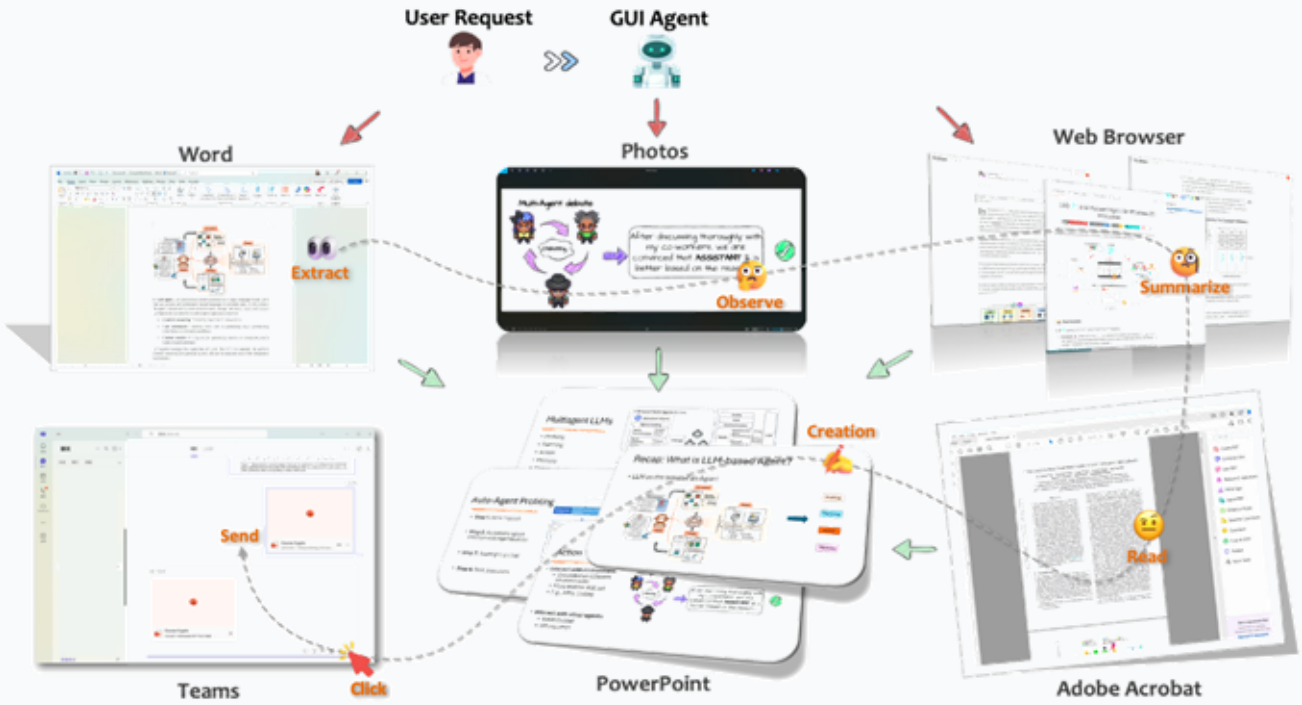



Core view: GUI agents translate natural-language intent into **executable operations** across real software interfaces.

Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

# Why do GUI agents matter now?

- Traditional GUI automation is brittle: scripts and rules work for fixed workflows but fail under dynamic layouts.
- LLMs/VLMs add three missing capabilities: **natural-language intent understanding**, **visual comprehension**, and **long-horizon reasoning**.
- The resulting agent can automate closed-source or legacy applications through the same visible interface used by humans.




Motivation distilled by the survey: **overcome rigid automation**, **democratize automation through natural commands**, and **enable real-world adoption** in productivity and accessibility.

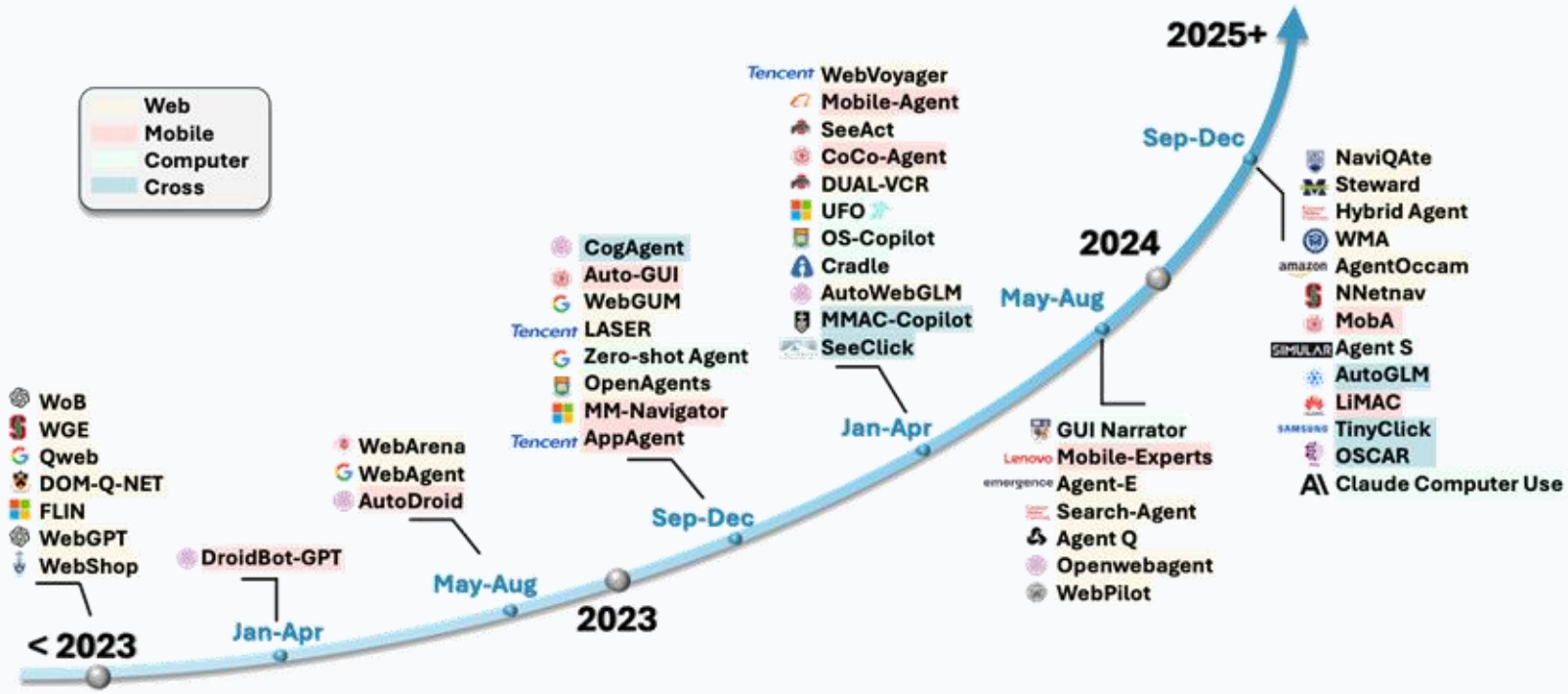
**From scripts**
Predefined, narrow, high-maintenance

→

**To agents**
Adaptive, multimodal, task-oriented

Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

# How did GUI agents evolve after LLMs?



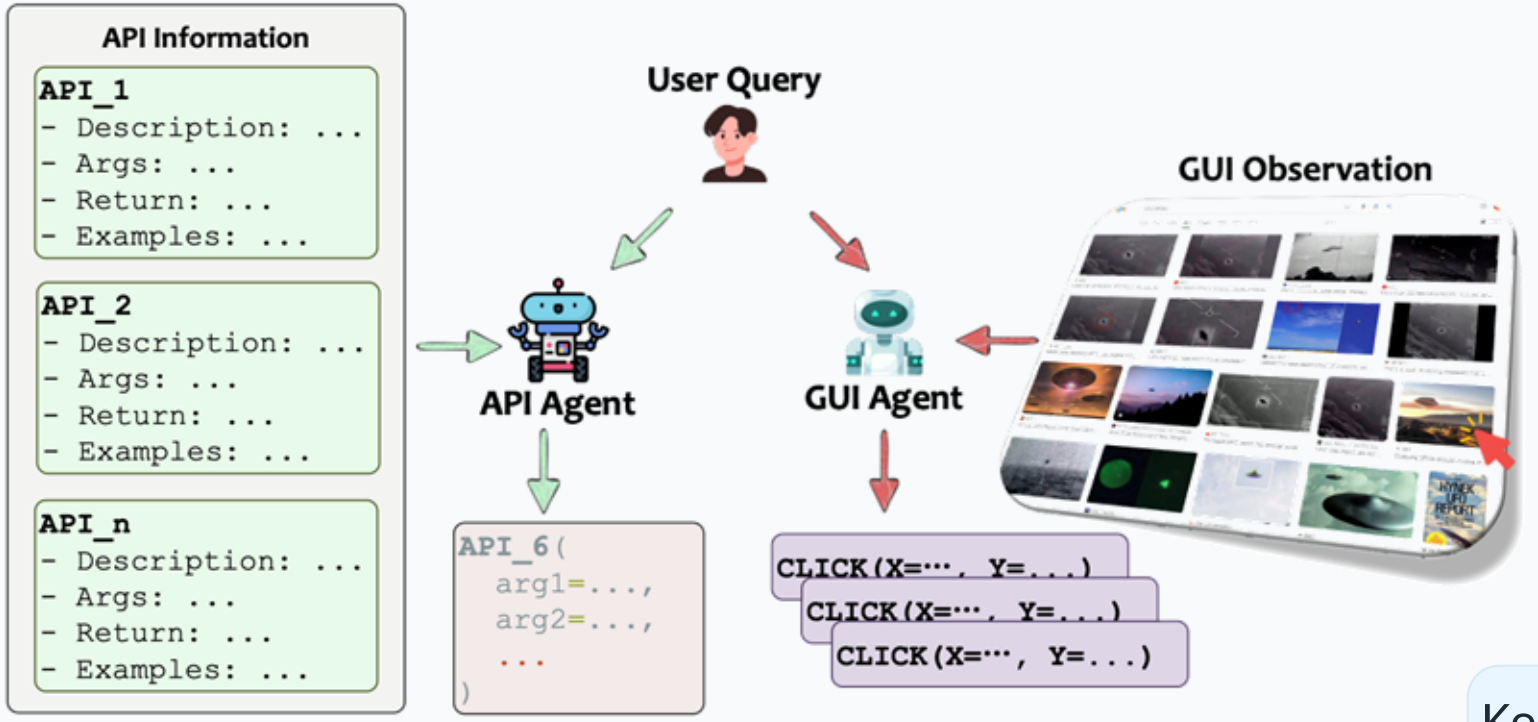

**< 2023**
Early GUI agents: mostly limited web testing or scripted automation.

**2023–2024**
Rapid emergence across web, mobile, computer, and cross-platform agents.

**2025+**
Shift toward OS-integrated agents, pure-vision grounding, and **large action models**.

Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

# What distinguishes GUI agents from API agents?




## GUI CONTROL

Universal action space: click, type, scroll, drag.
Works even when APIs are absent or inaccessible.
**Human-observable actions** improve transparency.

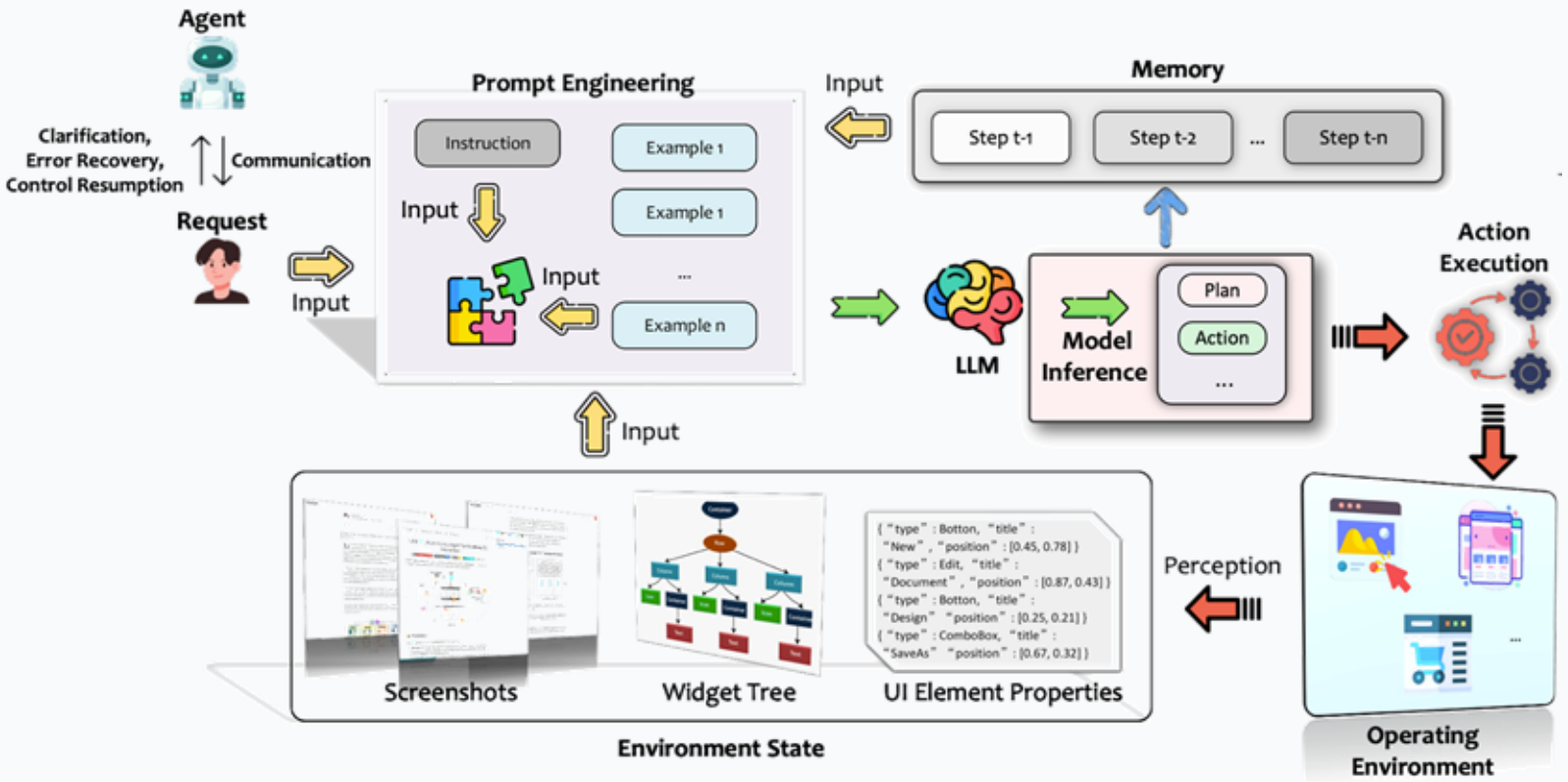
## API CONTROL

Direct function calls are **faster** and **less error-prone**.
But APIs can be private, limited, or platform-specific.
Best future direction is **hybrid control**: GUI fallback + API efficiency.

Key synthesis:
GUI agents maximize **coverage**;
API agents maximize **efficiency**.
Hybrid agents aim to combine both.

Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

# What is the core workflow of an LLM-brained GUI agent?




**1. Perceive**
Screenshot + widget tree + UI properties.

**2. Prompt**
Encode request, environment, actions, examples.

**3. Infer**
Plan and predict the next executable action.

**4. Act & update**
Execute action, read feedback, maintain memory.

The paper treats the agent as a loop, not a one-shot model call: **every action creates new evidence for the next step**.

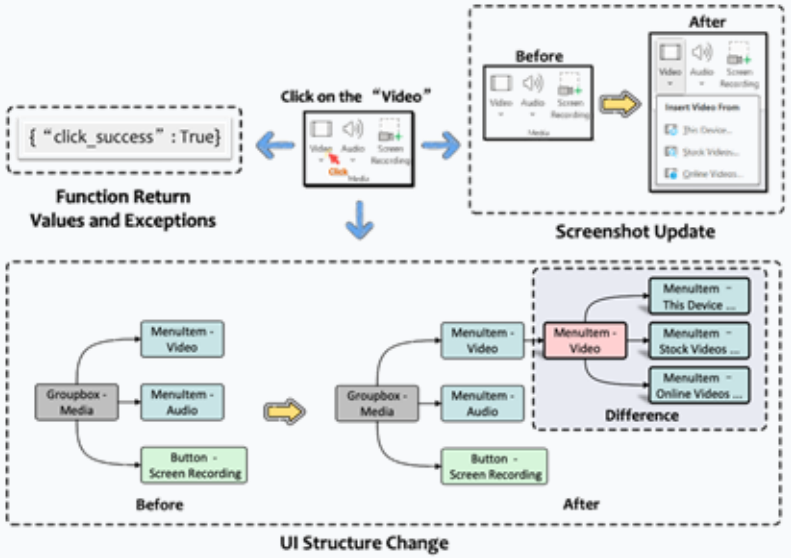
Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

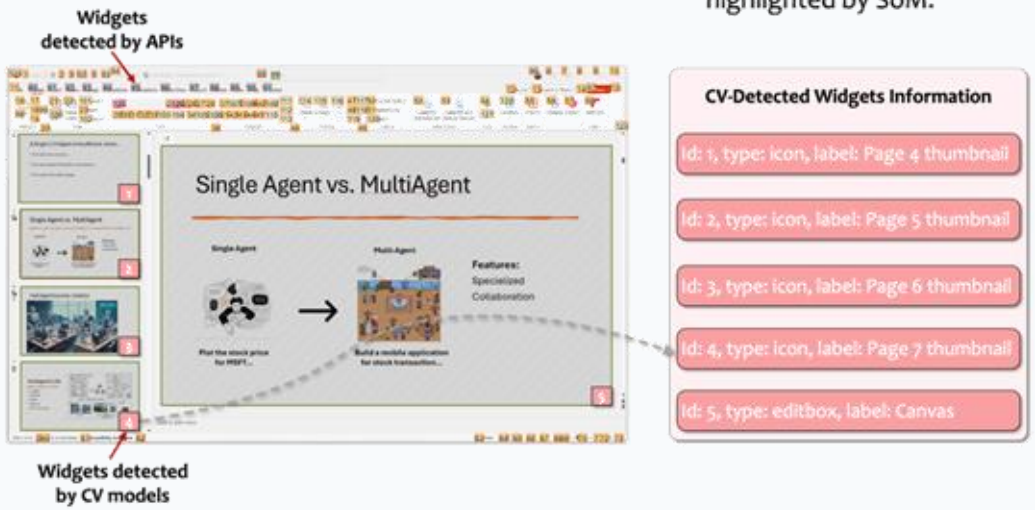
# What must the agent perceive from the GUI?

(a) A clean GUI screenshot.

(b) A GUI screenshot with widgets highlighted by SoM.

(c) A GUI screenshot with widgets highlighted by bounding boxes.



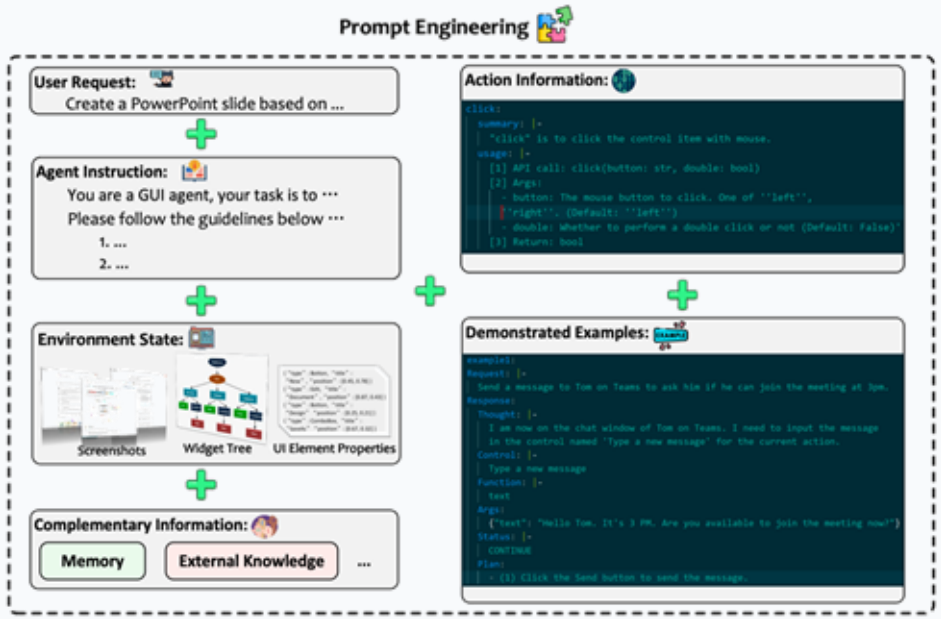



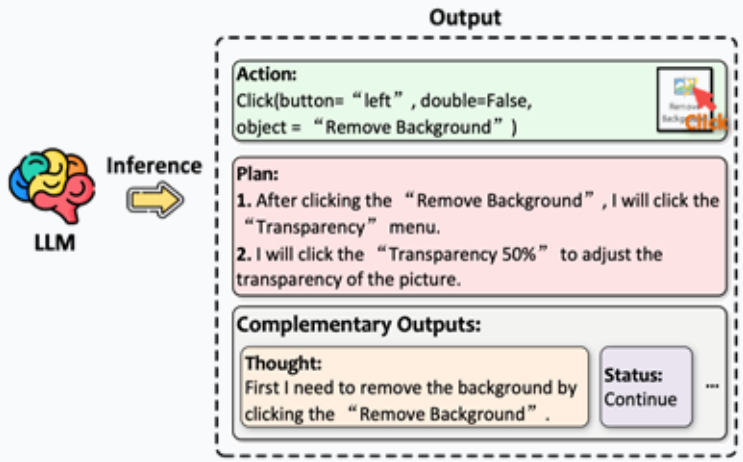

## PERCEPTION CHANNELS

**Screenshots** provide visual state and capture non-standard widgets.
**Widget trees** and **UI element properties** provide structured metadata when available.
**CV/OCR grounding** fills gaps when accessibility APIs are incomplete.
**Feedback** comes from screenshot updates, structure changes, and return values or errors.

Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

# How does the LLM turn a screen into an action?




**Prompt input**
Request + screenshot/widget data + action schema + examples + memory.

**LLM output**
Plan + action call + optional thoughts/status/user communication.

Design pressure: enough context to reason, but not so much context that latency and confusion dominate.

Source: Zhang et al., “Large Language Model-Brained GUI Agents: A Survey”, TMLR 2025

# What actions can the agent execute?

## 1 GUI operations

Mouse, keyboard, touch, gestures, clipboard, screen interactions.

*Broadest compatibility, but slow and error-prone for long workflows.*

## 2 Native API calls

Shell commands, app APIs, system APIs, web APIs.

*Fast and reliable when available, but less generalizable.*

## 3 AI tools

Summarization, generation, OCR, visual understanding, auxiliary agents.

*Adds capabilities beyond direct UI manipulation.*

The strongest agent does not rely on only one execution mode;
it chooses the most reliable action primitive for the current software and task.

Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

# What data and models drive specialized GUI agents?

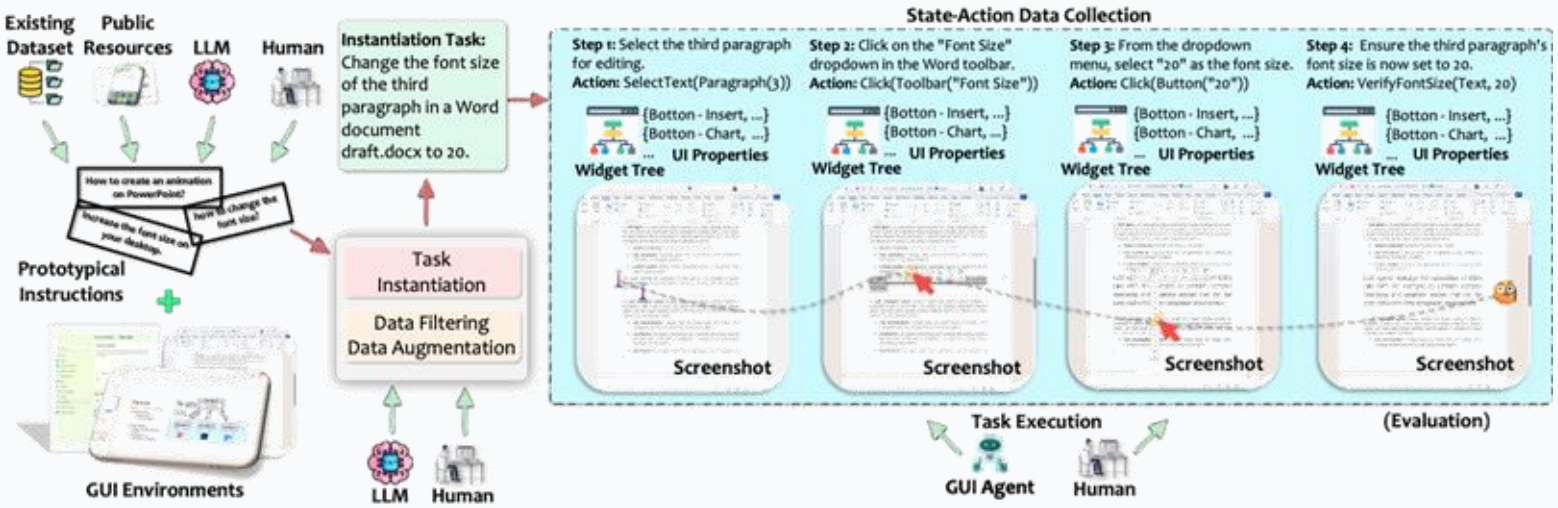



Figure 24: A complete pipeline for data collection for training a GUI agent model.

## DATA PIPELINE

Start from prototypical instructions sourced from humans, LLMs, public resources and existing datasets.
Instantiate tasks in concrete environments, collect state–action trajectories, then filter and augment.
High-quality GUI datasets require screenshots, metadata, actions and validation.

## MODEL DIRECTION

Foundation models provide general multimodal reasoning.
**L**arge **A**ction **M**odels specialize that reasoning with GUI-specific data.
Trends: smaller on-device models, stronger visual grounding, RL in dynamic environments, unified function calling, and inference-time reasoning.

The data–model loop is the engine of progress: better trajectories train better action models, and better models collect better trajectories.

Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

# How are GUI-agent frameworks organized?


Figure 6: Examples of GUIs from web, mobile and computer platforms.

Trend: the field is moving from platform-specific prototypes toward **modular, multimodal, and cross-platform agents**.

**Web agents**
SeeAct, WebVoyager, WebAgent, LASER, Search-Agent, WebPilot, WMA, WebDreamer, Hybrid Agent, AutoWebGLM

**Mobile agents**
AppAgent, Mobile-Agent, VisionTasker, DroidBot-GPT, CoCo-Agent, CoAT, AutoDroid, MobileGPT

**Computer agents**
UFO/UFO2, Cradle, OS-Copilot, Programming with Pixels

**Cross-platform**
AutoGLM, TinyClick, OSCAR, AgentStore, MMAC-Copilot, AGUVIS, Agent S2

Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

# How should GUI agents be evaluated and deployed?

## EVALUATION DIMENSIONS

**Task success**
Did the agent complete the user goal?

**Step efficiency**
How many actions, model calls, and tokens were required?

**Grounding accuracy**
Did it click or manipulate the intended element?

**Robustness & safety**
Can it recover from errors and avoid harmful actions?

## BENCHMARK FAMILIES

**Web**: MiniWoB++, WebArena, Mind2Web, VisualWebArena
**Mobile**: AITW, AndroidWorld and related app-control benchmarks
**Computer**: OSWorld, WindowsAgentArena and desktop automation suites
**Cross-platform**: VisualAgentBench, GUI-World and unified GUI datasets

## DEPLOYMENT AREAS

**GUI testing**: general testing, text-input generation, bug replay, verification
**Virtual assistants**: research prototypes, open-source projects, production products such as Power Automate, MultiOn, YOYO Agent and Eko

Evaluation must cover both **offline skills** and **live interaction**:
a model can locate buttons yet still fail a long-horizon user task.

Source: Zhang et al., “Large Language Model-Brained GUI Agents: A Survey”, TMLR 2025

# What challenges define the research roadmap?

### Privacy

Screenshots, credentials, documents and histories may contain sensitive data.

### Latency & cost

Long-horizon tasks amplify model calls, tokens and resource constraints.

### Safety & reliability

Agents need error recovery, permission control and safe execution boundaries.

### Human–agent interaction

Clarification, control handoff and transparency shape user trust.

### Generalization

Unseen apps, dynamic interfaces and personalization remain open problems.

### Ethics & regulation

Accountability, misuse prevention and compliance are necessary for deployment.

**Takeaway:** LLM-brained GUI agents are moving from demos to infrastructure, but the next leap depends on **trustworthy perception**, **efficient interaction**, and **safe human-centered control**.

Source: Zhang et al., "Large Language Model-Brained GUI Agents: A Survey", TMLR 2025

# GTA1: GUI TEST-TIME SCALING AGENT

**Yan Yang**[1] **Dongxu Li** ✉[2] **Yutong Dai**[1] **Yuhao Yang**[3] **Ziyang Luo**[1]
**Zirui Zhao**[1] **Zhiyuan Hu**[1] **Junzhe Huang**[2] **Amrita Saha**[1] **Zeyuan Chen**[1]
**Ran Xu**[1] **Liyuan Pan**[4] **Caiming Xiong** ✉[1] **Junnan Li** ✉[1]
[1]Salesforce AI Research [2]The Australian National University
[3]University of Hong Kong [4] Yangtze Delta Region Academy ✉Corresponding Author
dongxuli1005@gmail.com cxiong@salesforce.com junnan.li@salesforce.com

ICLR 2026

# GTA1

## GUI Test-Time Scaling Agent

**Can a GUI agent trade inference compute for more reliable computer use?**

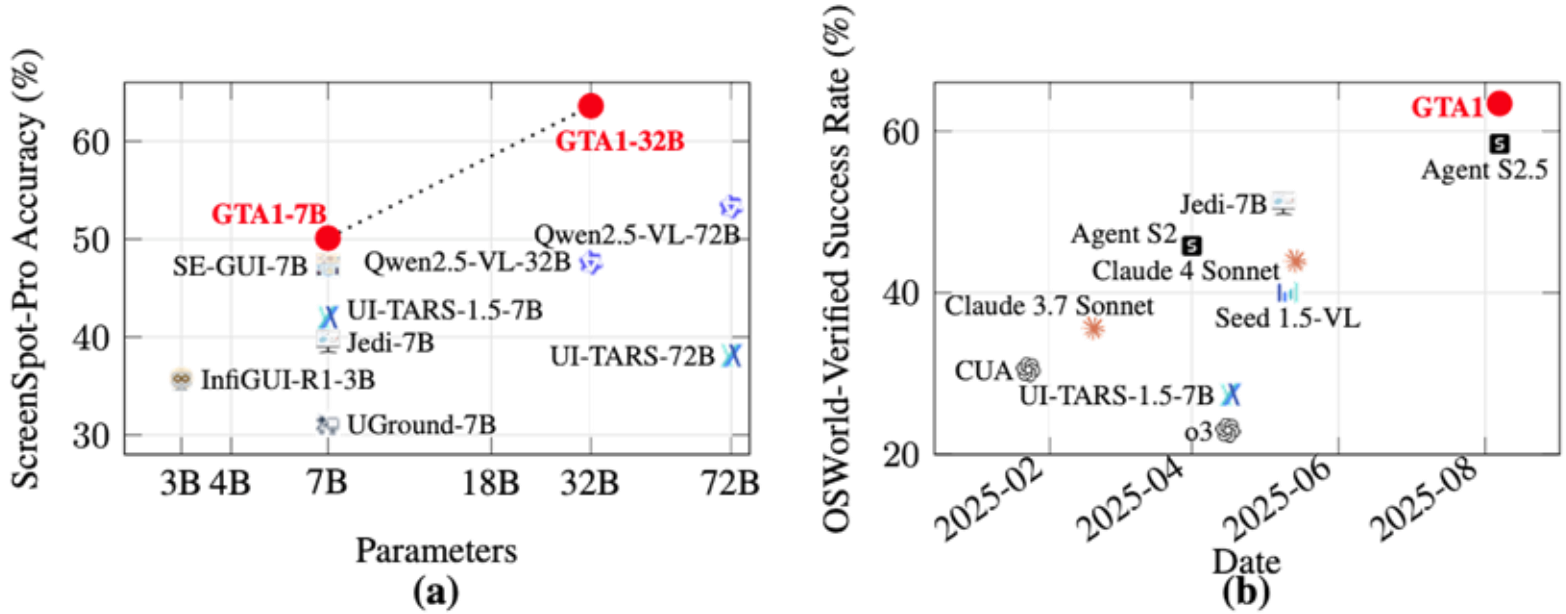



Figure 1: Comparison of GTA1 (red dot) with state-of-the-art methods. **(a)** Grounding accuracy (%) on the ScreenSpot-Pro benchmark (Li et al., 2025) across model scales in billions (B) of parameters. **(b)** Success rate (%) on the OSWorld-Verified benchmark (Xie et al., 2024) over time.

**GTA1 answers yes:**
**sample several next actions, judge them, then ground the chosen action precisely.**

| Planning uncertainty | Grounding precision | Task success |
|---|---|---|

Source: Yang et al., GTA1: GUI Test-Time Scaling Agent, ICLR 2026

# Why are GUI agents hard to make reliable?

**They must choose the right next step in a huge action space and click the right pixels on changing screens.**

User task → Many possible plans → No safe lookahead → Precise UI click

- **Planning**: multiple valid action sequences may exist, but early mistakes cascade.
- **Environment**: real GUI actions can be irreversible, so full rollout lookahead is impractical.
- **Grounding**: high-resolution professional interfaces require accurate coordinates.

**Core tension:**
**robust decisions require more exploration, but GUI execution is step-by-step.**

# Where do earlier GUI approaches leave room for improvement?

## SFT grounding overfits to element centers, while end-to-end native agents are powerful but hard to inspect.

**SFT grounding**

**Two-stage agents**

**Native/end-to-end agents**

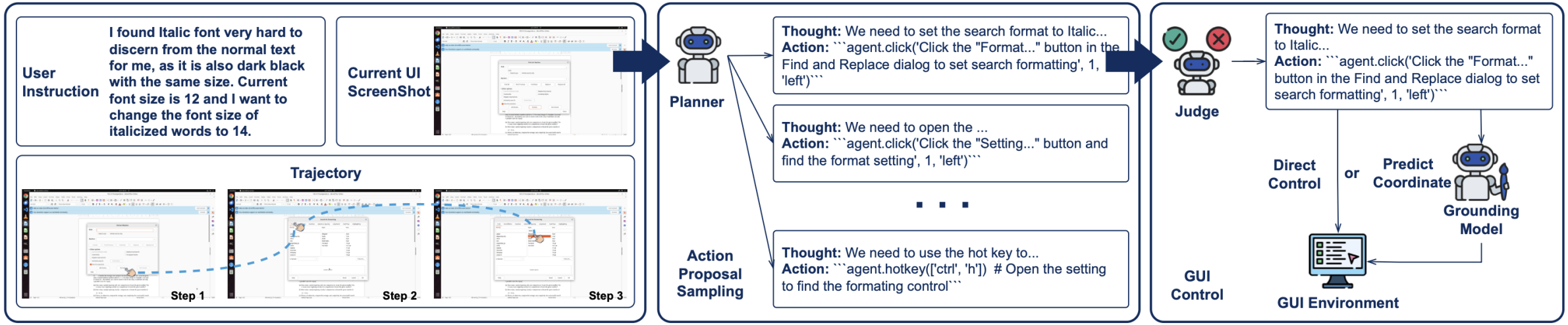
- Optimizes exact center points even though any coordinate inside the target element works.
- Separates planner and grounding model; easier to improve each module independently.
- Completes tasks end-to-end with perception, memory, planning, and action in one agent.

**GTA1 keeps the modular two-stage form, then strengthens both weak links: planning selection and coordinate grounding.**

Source: Yang et al., GTA1: GUI Test-Time Scaling Agent, ICLR 2026

# What is the core design of GTA1?

## A planner proposes actions; a judge selects one; a grounding model maps coordinate actions to pixels.




*Paper Fig. 2: planner samples candidate action proposals; judge selects; grounding model executes coordinate actions.*

- Coordinate-based actions, such as clicks, are sent to the grounding model.
- Non-coordinate actions, such as hotkeys or typing, can be executed directly.
- The loop repeats until task completion or termination.

# How does test-time scaling improve planning without lookahead?

**It explores local alternatives at each step rather than rolling out full trajectories.**

**1. Sample K proposals** → **2. Judge current fit** → **3. Execute one action**

**Context = user instruction + past trajectory + current screenshot**

- The judge evaluates alignment with user intent and current GUI state.
- Concurrency makes K proposals a compute-time tradeoff rather than a long sequential rollout.
- This reduces overcommitment to a single weak action proposal.

**Default in experiments: K = 8 action proposals per step.**

# Why clean grounding data before RL training?

**Click rewards need accurate target boxes; noisy boxes make correct clicks look wrong.**

- Use *OmniParser* to detect visible UI element boxes in each screenshot.
- Compare the annotated box with detected boxes using maximum IoU.
- Discard a sample when max IoU < τ; the paper uses τ = 0.3.


*Paper Fig. 3: annotation boxes can be misaligned with the rendered UI; cleaning filters them out.*

**Resulting training signal:**
**"inside the actual UI element" instead of "match a potentially wrong center point."**

# How is RL grounding aligned with real clicking?

**The model is rewarded for predicting any valid click point inside the target element.**

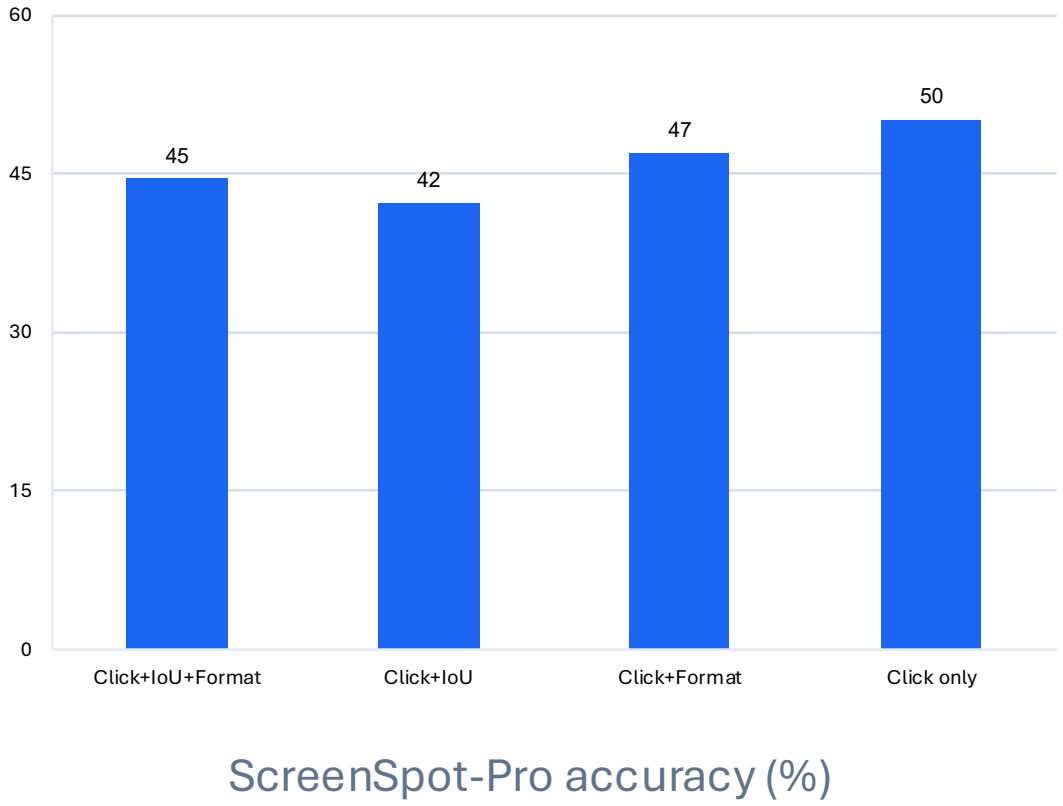
**Input: screenshot s + action proposal p** **Output: coordinate o = (x, y)**

$$\text{reward } r = 1 \text{ if } x_{min} \le x \le x_{max} \text{ and } y_{min} \le y \le y_{max}; \text{ otherwise } r = 0$$

- GRPO samples N responses per input and normalizes binary rewards into advantages.
- No mandatory chain-of-thought or bounding-box prediction is needed for static grounding.
- The objective matches GUI interaction: **clicking anywhere inside the target** should succeed.

# Which reward design works best for grounding?

## Click-only reward is the simplest and strongest overall design in the paper's ablation.



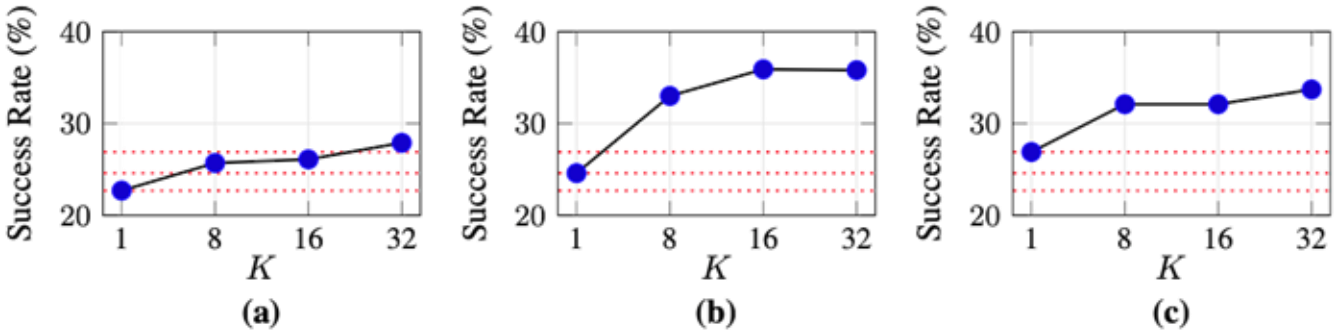

### Ablation lesson

- Bounding-box IoU reward adds extra prediction burden.
- Format reward enforces reasoning syntax but does not improve static grounding.
- **Click reward** directly matches the GUI success criterion.

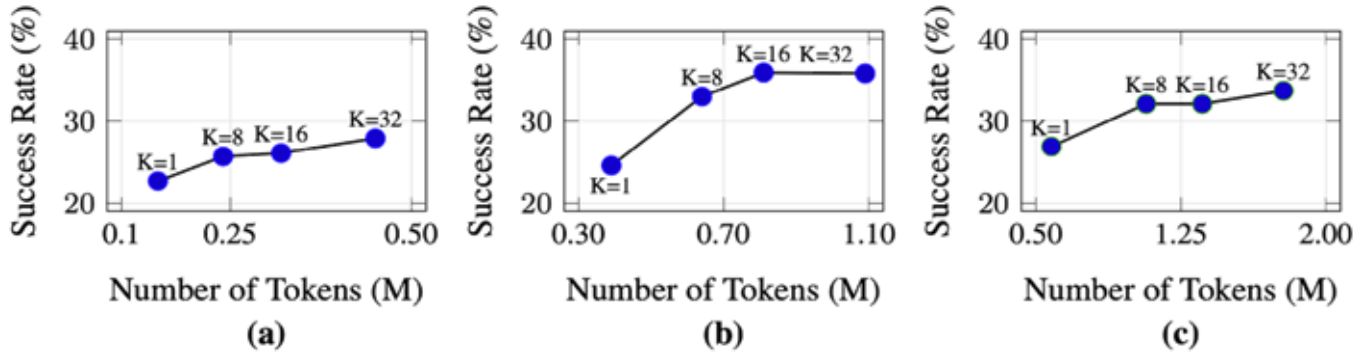
Source: Yang et al., GTA1: GUI Test-Time Scaling Agent, ICLR 2026

# How far should test-time scaling go?

## More candidate actions generally improve success, but token cost rises; the paper uses K = 8 by default.




*Paper Fig. 4: success rate increases with more candidate proposals K across step horizons.*



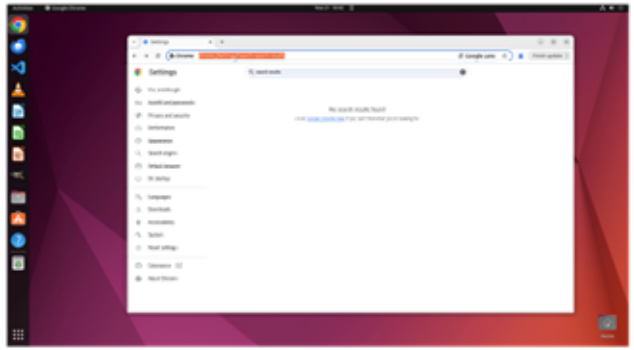

*Paper Fig. 5: higher K consumes more tokens per task, creating a compute-performance tradeoff.*

K = 1 is the no-scaling baseline.
K = 8/16/32 explores more local alternatives concurrently.
Best horizon is task-dependent: too few steps cannot finish; too many dilute the gain.

# What still fails, and what should we remember?

## GTA1 improves local decision quality, but a judge cannot select a correct action if none of the proposals are valid.

**User Instruction**: Could you please change the number of search results displayed on one page to 50? I find that having more results visible at once significantly enhances my research efficiency, as it reduces the need to constantly click through multiple pages.

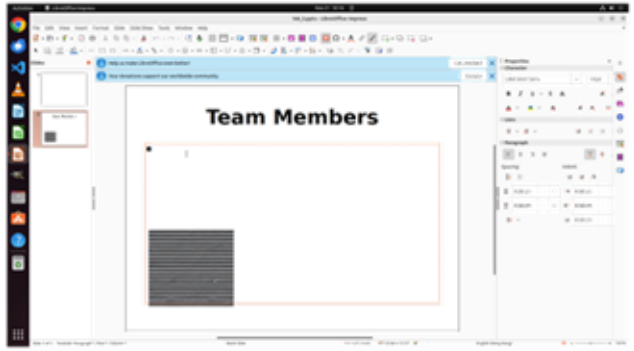


$p_1$: Previous attempts failed to find the desired settings, and I'm already in the address bar; it's time for a change of approach. Since it's a matter for Google's own search box, why not just go to their website? Typing google.com into the address bar should be the most direct way.

$p_2$: This search function seems a bit weak; I can't find an option to adjust the number of results displayed. I remember Chrome's settings page has a "Performance" option, which should involve some interface display settings. I'll check the left-hand menu first; maybe I can find the function I need there.

$p_3$: ...

**User Instruction**: On page 2, add a note same to the title into the slide. Make the font of title bold.




$p_1$: I noticed a "Click to add Text" area in the center of the slide, waiting for input. The key to completing the task is to add the title "Team Members" itself as a note. The cursor is now in the right position; I need to directly type this text to maintain consistency with the title.

$p_2$: I just clicked the input box to write a note, but that doesn't seem right. Looking at the menu bar, it seems more appropriate to add notes using the Insert function. Let me click the "Insert" option at the top to start this operation again.

$p_3$: ...

*Paper Fig. 7: failure cases occur when every sampled proposal is wrong or the task is infeasible.*

**Main contribution**:
test-time scaling for planning + click-reward RL for grounding.
**Key result**:
strong grounding and task-execution gains with a modular two-stage agent.
**Main limitation**:
proposal quality and task feasibility still bound the entire system.

**Takeaway:**
**robust GUI agents need both better "what to do next" selection and better "where to act" grounding.**